AgroRAG: asistente documental verificable para riego y estrés hídrico

Trabajo integrador - Procesamiento de Lenguaje Natural III

Autor y responsable académico: Gustavo Julián Rivas Versión técnica: 28/10/2026 Repositorio: https://github.com/gusjrivas/PLN3_AgroRAG_GJR

Estado del documento: entrega técnica reproducible con evaluación científica definitiva pendiente.

1. Objetivo, problema y alcance

AgroRAG es un asistente documental en español para consultar evidencia sobre riego y estrés hídrico en tomate y lechuga. El problema abordado no es predecir el riego ni controlar un sistema agrícola, sino recuperar pasajes verificables, construir una respuesta limitada por esos pasajes y mostrar la procedencia de cada afirmación. El proyecto prioriza trazabilidad, abstención y fallos operativos explícitos por encima de una respuesta aparentemente útil sin respaldo.

El alcance funcional incluye exploración lexical del corpus, recuperación semántica experimental, un flujo de investigador y verificador, API FastAPI, interfaz Streamlit, citas direccionables y auditoría redactada. Los estados públicos son `answered`, `needs_clarification` e `insufficient_evidence`. Los errores del proveedor, los timeouts y los contratos inválidos no se convierten en respuestas agronómicas.

Quedan fuera del MVP las predicciones con sensores, la automatización del riego, CNN o ViT, fine-tuning, búsqueda web libre durante una consulta, GraphRAG y despliegue Kubernetes. El sistema no prescribe litros de agua sin datos y evidencia suficientes. El corpus es piloto y no representa la totalidad de recomendaciones aplicables a una finca, región o campaña.

Criterios de éxito técnico

- Una respuesta aprobada sólo referencia `evidence_id` existentes en el catálogo activo.
- El frontend consume exclusivamente la API HTTP y no importa índices, agentes ni modelos.
- Los proveedores se inyectan desde bootstrap; importar módulos no descarga pesos ni abre red.
- Los secretos locales permanecen fuera de Git y nunca aparecen en auditoría o respuestas.
- Toda automatización nueva conserva evidencia RED, GREEN y REFACTOR.

2. Corpus y procedencia

El corpus piloto efectivo contiene 27 fragmentos revisados de cinco fuentes públicas. El repositorio versiona el manifiesto, las páginas seleccionadas, los hashes esperados y el procedimiento de descarga, pero no redistribuye los PDF. Una descarga cuyo hash no coincide se rechaza para evitar que una edición cambie silenciosamente el corpus.

Fuente	Selección	Uso	Limitación
INTA Huerta 2008	Lechuga, tomate y períodos críticos	Recomendaciones específicas de cultivo	Cobertura acotada y edición histórica
FAO56 1998	Evaporación, transpiración y agua disponible	Fundamentos técnicos	Texto en inglés y no específico de toda situación local
INTA Riego en cultivos hortícolas 2024	Agua disponible, programación y establecimiento	Fundamentos y operación de riego por goteo	Referencia regional del Oasis Norte de Mendoza
INTA Lechuga 2022	Suelo, salinidad, exceso de agua y programación	Cobertura específica de lechuga	Requiere interpretar cada recomendación en su contexto productivo
INTA Tomate 2020	Frecuencia, floración y cuajado	Cobertura específica de tomate	Hoja informativa breve y regional

Cada pasaje mantiene `source_id`, edición, URL, página física PDF, página impresa, sección, cultivo, texto original, texto adaptado y offsets. `corpus_version` deriva del manifiesto y de la configuración de extracción. El

catálogo SQLite se activa mediante un puntero pequeño y cada versión es inmutable. Una ingesta fallida conserva la versión anterior.

Las tablas seleccionadas se reconstruyen con encabezados, unidades, fila y contexto. Esta regla evita separar un número de su significado. Las páginas escaneadas sin texto se rechazan: el OCR no se activa como fallback silencioso. El manifiesto vigente debe consultarse para confirmar fechas, hashes y derechos antes de una reproducción.

Admisión reproducible

```
uv run agro-rag download data/manifest.json
uv run agro-rag ingest data/manifest.json --output data/processed/catalogue
```

La descarga requiere Internet y sólo admite los dominios institucionales configurados. La ingesta posterior es local. Los PDF y catálogos generados están ignorados por Git.

3. Arquitectura

La solución es un monolito modular de backend con una interfaz desplegada como proceso separado. El recorrido de una consulta es: usuario, Streamlit, API FastAPI, caso de uso, retrieval, investigador, verificador y respuesta tipada. La UI sólo conoce JSON por HTTP.

Capa	Responsabilidad	Restricción principal
domain	Evidencia y reglas puras	Sin frameworks ni proveedores
application	Casos de uso y puertos	Sin infraestructura, FastAPI o Streamlit
infrastructure	SQLite, BM25, OpenAI, Ollama, Pinecone, E5 y reranker	Implementa puertos, no reglas HTTP
api	Esquemas, autenticación y errores	Sin lógica agronómica
bootstrap	Configuración e inyección	Único ensamblaje de adaptadores
agro_rag_ui	Cliente HTTP y estado visual	No importa `agro_rag`

El investigador recibe pregunta, cultivo y evidencia. Puede producir claims, pedir aclaración o abstenerse. El verificador comprueba IDs y soporte, aprueba, solicita una revisión acotada o termina en aclaración/abstención. Hay un máximo de una corrección, seis llamadas LLM, cuatro operaciones de búsqueda, 12.000 tokens y 60 segundos globales. Retrieval sincrónico se despacha a un worker para no bloquear el event loop.

La API distingue liveness de readiness. Liveness no llama proveedores. Readiness requiere corpus y modelo configurados. Un semáforo limita dos consultas activas; el exceso devuelve 429. La caché LRU de retrieval usa consulta normalizada, cultivo y versión de corpus, nunca cachea errores ni respuestas generadas.

4. Implementación y reutilización

Backend y frontend usan Python 3.12. FastAPI expone salud, catálogo, evidencia y consultas; Streamlit presenta fuentes y respuesta verificada. Pydantic valida settings/HTTP y SQLite conserva el catálogo. BM25 es el default; vector, hybrid o hybrid_rerank requieren activación explícita e identidad, secreto y tokenizer compatibles. OpenAI y Ollama son proveedores HTTP opcionales.

Se adaptaron del código docente `Teoria1/Codigo/RAG`, commit `1f4cdbb37f59a049ef9b35c482ccb1de850fbc43`, los patrones de configuración, BM25, Pinecone, RRF y reranking. No se copiaron limpieza automática ni creación de índices en startup. El diseño separa fuente/página/pasaje y publica el manifiesto remoto sólo después de verificar IDs.

La integración semántica administrativa usa `intfloat/multilingual-e5-small`, revisión `614241f622f53c4eeff9890bdc4f31cfecc418b3`, con dimensión 384. El reranker experimental usa `BAAI/bge-reranker-v2-m3`, revisión `953dc6f6f85a1b2dbfca4c34a2796e7dde08d41e`. Ambos se cargan de forma diferida. `index-pinecone` requiere configuración completa y namespace nuevo; `--create-index` es explícito. El runtime verifica corpus/modelo/revisión/dimensión, cuenta contexto sin truncarlo con tokenizer fijado, no degrada a BM25 e informa el modo efectivo.

La implementación siguió TDD por escenarios. La suite contiene pruebas unitarias, contrato API, integración local, UI, arquitectura y navegador. Ruff, mypy, import-linter, OpenSpec y GitHub Actions forman las puertas de cada PR.

5. Metodología de evaluación

El protocolo preregistrado define un benchmark de 60 casos revisados: 36 respondibles, 12 sin soporte y 12 adversariales, con split agrupado 30 dev y 30 test. Cada split conserva 18/6/6 y una familia de paráfrasis nunca cruza entre ambos. Sólo dev puede usarse para elegir parámetros.

Cada caso registra pregunta, cultivo, categoría, estado esperado, respuesta de referencia, claims requeridos, evidencias relevantes, páginas fuente, anotador, revisión y versión de corpus. El validador implementado exige exactamente esa distribución, qrels resolubles y SHA-256 del JSONL. Los 36 casos respondibles integran promedios IR; los otros 24 se evalúan en abstención y aclaración, sin contaminar el denominador de recuperación.

Existe un paquete candidato de 60 casos con ese balance y evidencias resolubles contra el corpus activo. Todas sus filas están marcadas `pending_human`. Un validador separado comprueba el borrador sin permitir que el cargador científico lo acepte como `reviewed`; cada respuesta, claim, página e ID todavía debe contrastarse manualmente con la fuente.

Experimento	Variantes congeladas	Dimensión principal
E1	BM25, vector, hybrid, hybrid + reranker	Retrieval y latencia
E2	Sin verificador, con verificador	Soporte, citas y decisión del agente
E3	Caché fría, caché caliente	Latencia, tokens, llamadas y costo

El reporte EVA-02 exige consultas pareadas, errores completos, corpus/dataset/commit, versiones de prompts y modelos, tokens, p50/p95 y métricas separadas en retrieval, generación, agentes y operación. Una corrida `real` no puede mezclarse con `test_double` y exige revisión humana. Un resultado negativo se conserva junto con su denominador e IDs excluidos.

Las métricas previstas incluyen Precision@k, Recall@k, nDCG@10, MRR@10, cobertura de contexto, soporte factual, citas correctas, matriz de estados, decisiones del verificador, p50/p95, tokens, llamadas, fallos y costo con tarifa fechada. Similitud semántica puede complementar la revisión, pero no sustituye el soporte factual.

6. Resultados y evidencia

Verificación de ingeniería

Antes de cerrar este paquete, la suite local obtuvo `188 passed, 2 deselected`. Ruff, formato, mypy sobre 36 archivos, dos contratos de importación y los siete cambios OpenSpec finalizaron con código 0. Los jobs remotos `offline`, `specs` y `browser` del PR 22 también aprobaron. Estos datos demuestran consistencia técnica del repositorio, no calidad agronómica.

Smoke semántico preliminar

El 23/10/2026 se ejecutó una pasada real de retrieval sobre ocho preguntas de desarrollo y doce evidencias. Las etiquetas son `pending_human`, fueron creadas como borrador técnico y no forman el benchmark de 60 casos. Los valores sólo permiten detectar integración, costos y fallos.

Modo	MRR@10	Recall@5	nDCG@10	p95 ms
BM25	0,6875	0,750	0,7039	1,25
Vector E5	0,8438	1,000	0,8827	233,23
Hybrid RRF	0,8958	0,875	0,9195	268,01
Hybrid + BGE reranker	1,0000	1,000	1,0000	30559,45

El resultado conserva dos señales negativas. RRF redujo Recall@5 respecto del vector en este borrador. El reranker consumió aproximadamente 30,56 segundos p95 sólo en retrieval sobre CPU, incompatible con el objetivo de 30 segundos para la respuesta completa. La perfección aparente del reranker sobre ocho preguntas pequeñas no se generaliza y no autoriza activarlo.

No existen todavía resultados definitivos de E1, E2 o E3. El paquete candidato de 60 casos no fue revisado por una persona, no se midió soporte factual final y no se ejecutaron tres pasadas fría/caliente. Por eso la API pública permanece en BM25 y este informe no declara mejora científica.

7. Seguridad, operación y costos

`.env` y todas sus variantes privadas están ignoradas; `.env.example` sólo contiene nombres y valores no secretos. La clave Pinecone se lee como `SecretStr` y se entrega explícitamente al adaptador. La alternativa más segura para una sesión es una variable de proceso obtenida mediante entrada segura de PowerShell. En CI se requiere un secret store.

La API usa `X-API-Key`, validación de tamaño y esquema, lista cerrada de estados, límites de concurrencia y errores redactados. La descarga de corpus sólo existe en CLI administrativa y restringe dominios, redirects, rutas y hashes. Startup no crea ni borra índices.

La auditoría registra request ID, versión del corpus, decisión, revisiones, duración, llamadas, tokens, estimación y cache hit. No registra pregunta, prompt, evidencia, URL, clave o mensaje de excepción. El request ID del error HTTP coincide con el evento interno sin exponer detalles.

No se informa un costo económico final porque las corridas definitivas no se realizaron. OpenAI, Pinecone y las descargas pueden generar uso externo; el operador debe revisar su cuenta y presupuesto. Ollama se ejecuta localmente. El reporte EVA-02 admite `estimated_cost` separado y obliga a fechar cualquier tarifa.

8. Reproducción

Requisitos: Git, Python 3.12, uv y PowerShell o un shell equivalente. Desde un checkout limpio:

```
uv sync --frozen
Copy-Item .env.example .env
uv run agro-rag download data/manifest.json
uv run agro-rag ingest data/manifest.json --output data/processed/catalogue
uv run pytest -q
```

Luego se inician backend y frontend en terminales separadas con los comandos de `docs/run-local.md`. La exploración de fuentes funciona sin LLM. La respuesta verificada requiere OpenAI u Ollama y un modelo configurado. Pinecone y el extra semántico son opcionales y no forman parte del arranque público.

El informe se regenera con:

```
uv run python scripts/build_report.py
```

El verificador offline calcula hashes de los artefactos, comprueba comandos, secciones, duración del guion y presencia del PDF sin leer `.env` ni llamar a Internet. La release definitiva sólo puede etiquetarse después de incorporar el benchmark humano, ejecutar E1/E2/E3 y repetir la instalación en un checkout limpio.

9. Demo de 15 minutos

El guion versionado divide la exposición en problema/corpus, arquitectura/reuso, demo, evaluación y conclusiones. La demo muestra liveness, exploración de fuentes, una respuesta verificada cuando el proveedor configurado está disponible y un caso de aclaración o abstención. No se presenta un fixture como salida en vivo. Si un proveedor externo falla, se muestra el error real y se continúa con evidencia capturada e identificada como respaldo.

La secuencia exacta, preguntas, comandos, criterios de éxito y plan de contingencia están en `docs/demo.md`. Se recomienda ensayar con cronómetro y registrar hardware, modelo y commit antes de la exposición.

10. Modalidad y responsabilidades humanas

El autor y responsable académico registrado es Gustavo Julián Rivas. La modalidad individual o grupal todavía debe confirmarse con la cátedra. Si se confirma un grupo, esta sección debe ampliarse antes de entregar con integrantes, tareas efectivamente realizadas y revisión humana. Los agentes de programación utilizados no sustituyen responsables humanos ni se presentan como integrantes.

Las decisiones de arquitectura, aceptación de resultados, uso de credenciales, revisión de qrels y envío académico corresponden a una persona responsable. Ningún correo ni publicación se realiza de forma automática.

11. Conclusiones y trabajo pendiente

AgroRAG alcanza una primera versión local modular: corpus con procedencia, búsqueda lexical, API, UI, dos roles acotados, citas verificables, seguridad básica, caché y auditoría redactada. Los adaptadores semánticos se integraron en un namespace aislado y demostraron que el pipeline puede medirse. El runtime permite una activación explícita y fail-closed para corridas controladas, sin cambiar BM25 como default ni confundir capacidad técnica con validación científica.

La principal conclusión técnica es que mayor score preliminar no implica una mejor decisión de producto. El reranker fue demasiado lento y los qrels no tienen revisión humana. Mantener BM25 en la API es una decisión conservadora sustentada por evidencia, no una omisión silenciosa.

Para cerrar científicamente el trabajo faltan: revisar humanamente los 60 casos candidatos, congelar dev/test, ejecutar E1/E2/E3 con proveedores reales, revisar soporte factual y citas, repetir mediciones, registrar hardware/costo y actualizar este PDF con resultados definitivos. También debe confirmarse la fecha calendario de Teoría 7 y la modalidad académica.

12. Referencias

- Repositorio docente PLN3, commit `1f4cdbb37f59a049ef9b35c482ccb1de850fbc43`, consigna y código RAG: https://github.com/gusjrivas/PLN3_GJR
- Goites, E. D. Manual de cultivos para la huerta orgánica familiar. INTA, 2008.
- Allen, R. G.; Pereira, L. S.; Raes, D.; Smith, M. Crop evapotranspiration. FAO Irrigation and Drainage Paper 56, 1998.
- Multilingual E5 small: https://huggingface.co/intfloat/multilingual-e5-small
- BGE reranker v2 m3: https://huggingface.co/BAAI/bge-reranker-v2-m3
- FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/
- Streamlit AppTest: https://docs.streamlit.io/develop/api-reference/app-testing

13. Declaración de asistencia de IA

Se utilizó Codex como asistente de desarrollo para explorar el repositorio, proponer cambios, escribir y ejecutar pruebas, implementar código, revisar documentación y preparar este informe. El proceso quedó sujeto a instrucciones versionadas, TDD, revisión de diffs, checks locales y CI. También se utilizó Engram como memoria local de desarrollo cuando estuvo disponible.

La asistencia de IA no constituye revisión agronómica, anotación humana del benchmark ni aprobación académica. Las métricas `pending_human`, las limitaciones y los resultados negativos se mantienen explícitos. La persona responsable debe revisar el contenido, validar los qrels y decidir la entrega final.